

Министерство науки и высшего образования Российской Федерации  
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ  
ВЫСШЕГО ОБРАЗОВАНИЯ  
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ**  
**ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5**  
**«Процедуры, функции, триггеры в PostgreSQL»**  
по дисциплине «Проектирование и реализация баз данных»

Обучающийся Остапишин Н.А.  
Факультет прикладной информатики  
Группа К3241  
Направление подготовки 09.03.03 Прикладная информатика  
Образовательная программа Мобильные и сетевые технологии  
Преподаватель Говорова Марина Михайловна

Санкт-Петербург  
2025/2026

## 1. Цель работы

Овладеть практическими навыками создания и использования процедур, функций и триггеров в базе данных PostgreSQL для базы данных ресторана.

## 2. Практическое задание

Для индивидуальной базы данных «Ресторан» необходимо создать хранимые процедуры, функции и триггеры. Работа выполняется в SQL Shell psql.

1. Создать три хранимые процедуры для индивидуальной БД согласно варианту.
2. Создать необходимые триггеры для индивидуальной БД.
3. Проверить работу процедур и триггеров с помощью SQL-запросов.
4. Сделать подтверждающие скриншоты выполнения в psql.

### Задание 4. Создать хранимые процедуры:

- Вывести сведения о заказах заданного официанта на заданную дату.
- Выполнить расчет стоимости заданного заказа.
- Повышение оклада заданного сотрудника на 30% при повышении его категории.

### Задание 5. Создать необходимые триггеры.

## 3. Выполнение работы

### Рисунок: Подключение к базе данных ресторана



```
postgres=# \c restoran
Вы подключены к базе данных "restoran" как пользователь "postgres".
restoran=# |
```

### 3.1. Процедура вывода заказов официанта на дату

Процедура принимает идентификатор официанта и дату, после чего выводит в NOTICE все заказы этого официанта на заданную дату. Официанты хранятся в таблице employee, а заказы — в таблице order с полем employee\_id.

```
CREATE OR REPLACE PROCEDURE pr_waiter_orders(
    p_employee_id INT,
    p_date DATE
)
LANGUAGE plpgsql
AS $$
BEGIN
    -- Выводим сведения о заказах официанта на заданную дату
    RAISE NOTICE '=== Заказы официанта (id=%) на дату % ===',
        p_employee_id, p_date;

    RAISE NOTICE 'order_id | table_id | order_time | status | cost';

    PERFORM 1
    FROM public."order" o
    WHERE o.employee_id = p_employee_id
        AND o.order_date = p_date;

END;
```

```
$$;
```

### Рисунок: Создание процедуры pr\_waiter\_orders, сообщение CREATE PROCEDURE

```
restoran=# CREATE OR REPLACE PROCEDURE pr_waiter_orders(  
restoran(#      p_employee_id INT,  
restoran(#      p_date DATE  
restoran(# )  
restoran=# LANGUAGE plpgsql  
restoran=# AS $$  
restoran$$ BEGIN  
restoran$$      -- Выводим сведения о заказах официанта на заданную дату  
restoran$$      RAISE NOTICE '=== Заказы официанта (id=%) на дату % ===',  
restoran$$          p_employee_id, p_date;  
restoran$$      RAISE NOTICE 'order_id | table_id | order_time | status | cost';  
restoran$$      PERFORM 1  
restoran$$      FROM public."order" o  
restoran$$      WHERE o.employee_id = p_employee_id  
restoran$$          AND o.order_date = p_date;  
restoran$$  
restoran$$ END;  
restoran$$ $$;  
CREATE PROCEDURE  
restoran=# |
```

#### Проверка до и после выполнения процедуры:

Проверка: смотрим какие заказы есть у официанта id=2

```
SELECT  
    o.id          AS order_id,  
    o.table_id,  
    o.order_date,  
    o.order_time,  
    o.status,  
    o.cost,  
    e.last_name || ' ' || e.first_name AS waiter  
FROM public."order" o  
JOIN public.employee e ON e.id = o.employee_id  
WHERE o.employee_id = 2  
      AND o.order_date = '2026-04-22';
```

Вызов процедуры

```
CALL pr_waiter_orders(2, '2026-04-22');
```

## 3.2. Процедура расчёта стоимости заказа (pr\_calculate\_order\_cost)

Процедура принимает идентификатор заказа, вычисляет суммарную стоимость всех позиций (цена блюда × количество) и записывает результат в поле cost таблицы order. Предварительно добавляется столбец price в таблицу dish.

Процедура расчёта стоимости заказа

```
CREATE OR REPLACE PROCEDURE pr_calculate_order_cost(  
    p_order_id INT  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_total NUMERIC(10,2);  
BEGIN  
    SELECT COALESCE(SUM(d.price * oi.quantity), 0)
```

```

        INTO v_total
        FROM public.order_item oi
        JOIN public.dish d ON d.id = oi.dish_id
        WHERE oi.order_id = p_order_id;

        UPDATE public."order"
        SET cost = v_total
        WHERE id = p_order_id;

        RAISE NOTICE 'Заказ №%: стоимость рассчитана и записана = % руб.',
            p_order_id, v_total;
END;
$$;

```

### Проверка до и после вызова процедуры:

Проверка до вызова процедуры

```
SELECT id, cost FROM public."order" WHERE id = 1;
```

Вызов процедуры

```
CALL pr_calculate_order_cost(1);
```

Проверка после вызова

```
SELECT id, cost FROM public."order" WHERE id = 1;
```

## 3.3. Процедура повышения оклада сотрудника на 30% (pr\_promote\_employee)

Процедура принимает идентификатор сотрудника, находит его активную должность (end\_date IS NULL), увеличивает оклад на 30% и повышает категорию на 1. Предварительно добавляется столбец category в таблицу employee\_position.

```
ALTER TABLE public.employee_position
    ADD COLUMN IF NOT EXISTS category INT DEFAULT 1 NOT NULL;
```

Процедура повышения оклада сотрудника на 30% при повышении категории

```
CREATE OR REPLACE PROCEDURE pr_promote_employee(
    p_employee_id INT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_old_salary NUMERIC(10,2);
    v_old_cat     INT;
    v_new_salary NUMERIC(10,2);
    v_new_cat     INT;
BEGIN
    SELECT salary, category
    INTO v_old_salary, v_old_cat
    FROM public.employee_position
    WHERE employee_id = p_employee_id
        AND end_date IS NULL
    LIMIT 1;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Активная должность для сотрудника id=% не найдена',
            p_employee_id;
    END IF;

    v_new_salary := ROUND(v_old_salary * 1.30, 2);
    v_new_cat    := v_old_cat + 1;

```

```

UPDATE public.employee_position
SET salary = v_new_salary,
    category = v_new_cat
WHERE employee_id = p_employee_id
AND end_date IS NULL;

RAISE NOTICE 'Сотрудник id=%: категория % -> %, оклад % -> % руб.',
    p_employee_id, v_old_cat, v_new_cat, v_old_salary, v_new_salary;
END;
$$;

```

## Проверка до и после вызова процедуры:

Проверка до повышения

```

SELECT ep.employee_id, e.last_name, ep.salary, ep.category
FROM public.employee_position ep
JOIN public.employee e ON e.id = ep.employee_id
WHERE ep.employee_id = 3 AND ep.end_date IS NULL;

```

Вызов процедуры

```
CALL pr_promote_employee(3);
```

Проверка после повышения

```

SELECT ep.employee_id, e.last_name, ep.salary, ep.category
FROM public.employee_position ep
JOIN public.employee e ON e.id = ep.employee_id
WHERE ep.employee_id = 3 AND ep.end_date IS NULL;

```

## 4. Триггеры

### 4.1. Триггер запрета изменения статуса завершённого заказа

Триггер не позволяет менять поле status у заказа, если он уже находится в состоянии completed, paid или cancelled. Срабатывает BEFORE UPDATE на таблице order.

Запрет изменения статуса завершённого заказа

```

CREATE OR REPLACE FUNCTION fn_prevent_completed_order_change()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF OLD.status IN ('completed', 'paid', 'cancelled') THEN
        RAISE EXCEPTION
            'Нельзя изменить статус заказа №% он уже "%"',
            OLD.id, OLD.status;
    END IF;
    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_prevent_completed_order_change ON public."order";

CREATE TRIGGER trg_prevent_completed_order_change
BEFORE UPDATE ON public."order"
FOR EACH ROW
EXECUTE PROCEDURE fn_prevent_completed_order_change();

```

### Тестирование триггера:

Тест: попытка изменить статус завершенного заказа (order id=1, status='completed')  
UPDATE public."order" SET status = 'active' WHERE id = 1;  
Ожидается ошибка: ERROR: Нельзя изменить статус заказа №1 – он уже "completed"

Тест успешного обновления активного заказа  
UPDATE public."order" SET status = 'completed' WHERE id = 2;

## 4.2. Триггер автоматического пересчёта стоимости заказа

Триггер срабатывает AFTER INSERT / UPDATE / DELETE на таблице order\_item и автоматически пересчитывает поле cost в таблице order как сумму price × quantity для всех позиций заказа.

Автоматический пересчёт стоимости заказа при добавлении/удалении позиции

```
CREATE OR REPLACE FUNCTION fn_recalc_order_cost()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_order_id INT;
    v_total     NUMERIC(10,2);
BEGIN

    IF TG_OP = 'DELETE' THEN
        v_order_id := OLD.order_id;
    ELSE
        v_order_id := NEW.order_id;
    END IF;

    SELECT COALESCE(SUM(d.price * oi.quantity), 0)
    INTO v_total
    FROM public.order_item oi
    JOIN public.dish d ON d.id = oi.dish_id
    WHERE oi.order_id = v_order_id;

    UPDATE public."order"
    SET cost = v_total
    WHERE id = v_order_id;

    RETURN COALESCE(NEW, OLD);
END;
$$;

DROP TRIGGER IF EXISTS trg_recalc_order_cost ON public.order_item;

CREATE TRIGGER trg_recalc_order_cost
AFTER INSERT OR UPDATE OR DELETE ON public.order_item
FOR EACH ROW
EXECUTE PROCEDURE fn_recalc_order_cost();
```

### Тестирование триггера:

Проверка ДО: стоимость заказа 4  
SELECT id, cost FROM public."order" WHERE id = 4;

Добавляем позицию в заказ 4  
INSERT INTO public.order\_item (order\_id, dish\_id, quantity, cooking\_status)  
VALUES (4, 3, 1, 'pending');

Проверка ПОСЛЕ: cost должен обновиться автоматически  
SELECT id, cost FROM public."order" WHERE id = 4;

### 4.3. Триггер запрета совмещения ролей официанта и повара в одном заказе

Триггер проверяет при INSERT или UPDATE строки order\_item, что назначаемый повар (cook\_id) не является официантом данного заказа (employee\_id в таблице order). Это исключает конфликт ролей.

Запрет назначения официанта (employee\_id) поваром в order\_item

```
CREATE OR REPLACE FUNCTION fn_check_cook_is_not_waiter()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_count INT;
BEGIN
    IF NEW.cook_id IS NULL THEN
        RETURN NEW;
    END IF;

    SELECT COUNT(*)
    INTO v_count
    FROM public."order" o
    WHERE o.id = NEW.order_id
        AND o.employee_id = NEW.cook_id;

    IF v_count > 0 THEN
        RAISE EXCEPTION
        'Сотрудник id=% является официантом заказа №% и не может быть назначен
поваром',
        NEW.cook_id, NEW.order_id;
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_check_cook_is_not_waiter ON public.order_item;

CREATE TRIGGER trg_check_cook_is_not_waiter
BEFORE INSERT OR UPDATE ON public.order_item
FOR EACH ROW
EXECUTE PROCEDURE fn_check_cook_is_not_waiter();
```

#### Тестирование триггера:

Заказ id=1 обслуживает официант employee\_id=2 (Петрова)  
Попытка назначить того же сотрудника поваром  
INSERT INTO public.order\_item (order\_id, dish\_id, cook\_id, quantity, cooking\_status)

```
VALUES (1, 6, 2, 1, 'pending');
```

Ожидается ошибка: ERROR: Сотрудник id=2 является официантом заказа №1 ...

Корректная вставка – другой сотрудник

```
INSERT INTO public.order_item (order_id, dish_id, cook_id, quantity, cooking_status)
VALUES (1, 6, 3, 1, 'pending');
```

#### 4.4. Триггер запрета удаления блюда, присутствующего в активных заказах

Триггер срабатывает BEFORE DELETE на таблице dish и блокирует удаление блюда, если оно входит хотя бы в один активный заказ (status = 'active'). Это защищает целостность текущих заказов.

Запрет удаления блюда, если оно присутствует в активных заказах

```
CREATE OR REPLACE FUNCTION fn_prevent_dish_delete_if_active()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_count INT;
BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM public.order_item oi
    JOIN public."order" o ON o.id = oi.order_id
    WHERE oi.dish_id = OLD.id
        AND o.status = 'active';

    IF v_count > 0 THEN
        RAISE EXCEPTION
            'Нельзя удалить блюдо "%" (id=%): оно присутствует в % активных заказах',
            OLD.name, OLD.id, v_count;
    END IF;

    RETURN OLD;
END;
$$;
```

```
DROP TRIGGER IF EXISTS trg_prevent_dish_delete_if_active ON public.dish;
```

```
CREATE TRIGGER trg_prevent_dish_delete_if_active
BEFORE DELETE ON public.dish
FOR EACH ROW
EXECUTE PROCEDURE fn_prevent_dish_delete_if_active();
```

#### Тестирование триггера:

Блюдо id=1 ("Цезарь с курицей") есть в активном заказе id=4

Попытка удалить блюдо

```
DELETE FROM public.dish WHERE id = 1;
```

Ожидается ошибка: ERROR: Нельзя удалить блюдо "Цезарь с курицей" (id=1): ...

Блюдо id=6 (Греческий салат) – тоже в активном заказе 4; удалим безопасное:

Сначала убедимся, что блюдо не в активных заказах, и тогда удалим

```
DELETE FROM public.dish WHERE id = 6
AND NOT EXISTS (
    SELECT 1 FROM public.order_item oi
    JOIN public."order" o ON o.id = oi.order_id
    WHERE oi.dish_id = 6 AND o.status = 'active'
```



```
);
```

#### 4.5. Триггер автоматического обновления остатка ингредиента при поставке

Триггер срабатывает AFTER INSERT OR DELETE на таблице supply\_ingredient и автоматически корректирует поле stock\_quantity: при новой поставке — увеличивает, при отмене — уменьшает (но не ниже нуля).

Автоматическое обновление stock\_quantity при поставке ингредиента

```
CREATE OR REPLACE FUNCTION fn_update_stock_on_supply()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- При добавлении новой поставки увеличиваем остаток
    IF TG_OP = 'INSERT' THEN
        UPDATE public.ingredient
        SET stock_quantity = stock_quantity + NEW.quantity
        WHERE id = NEW.ingredient_id;

    -- При удалении поставки уменьшаем остаток (не ниже 0)
    ELIF TG_OP = 'DELETE' THEN
        UPDATE public.ingredient
        SET stock_quantity = GREATEST(stock_quantity - OLD.quantity, 0)
        WHERE id = OLD.ingredient_id;
    END IF;

    RETURN COALESCE(NEW, OLD);
END;
$$;

DROP TRIGGER IF EXISTS trg_update_stock_on_supply ON public.supply_ingredient;

CREATE TRIGGER trg_update_stock_on_supply
AFTER INSERT OR DELETE ON public.supply_ingredient
FOR EACH ROW
EXECUTE PROCEDURE fn_update_stock_on_supply();
```

#### Тестирование триггера:

Проверка ДО: остаток ингредиента id=1 (Куриное филе)  
SELECT id, name, stock\_quantity FROM public.ingredient WHERE id = 1;

Добавляем поставку 5 кг  
INSERT INTO public.supply\_ingredient  
(ingredient\_id, purchase\_date, quantity, price, supplier)  
VALUES (1, CURRENT\_DATE, 5.000, 350.00, 'Мясная Лавка');

Проверка ПОСЛЕ: stock\_quantity должен увеличиться на 5  
SELECT id, name, stock\_quantity FROM public.ingredient WHERE id = 1;

#### 4.6. Триггер проверки количества в позиции заказа (запрет <= 0)

Триггер срабатывает BEFORE INSERT OR UPDATE на таблице order\_item и запрещает добавление или изменение позиции заказа с нулевым или отрицательным количеством блюд.

```
-- ТРИГГЕР 6: Запрет установки quantity <= 0 в order_item

CREATE OR REPLACE FUNCTION fn_check_order_item_quantity()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.quantity <= 0 THEN
        RAISE EXCEPTION
            'Количество блюда в заказе должно быть больше нуля. Передано: %',
            NEW.quantity;
    END IF;
    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_check_order_item_quantity ON public.order_item;

CREATE TRIGGER trg_check_order_item_quantity
BEFORE INSERT OR UPDATE ON public.order_item
FOR EACH ROW
EXECUTE PROCEDURE fn_check_order_item_quantity();
```

### Тестирование триггера:

```
Попытка добавить позицию с quantity = 0
INSERT INTO public.order_item (order_id, dish_id, quantity, cooking_status)
VALUES (2, 5, 0, 'pending');
Ожидается ошибка: ERROR: Количество блюда в заказе должно быть больше нуля. Передано:
0
```

```
Попытка с отрицательным quantity
INSERT INTO public.order_item (order_id, dish_id, quantity, cooking_status)
VALUES (2, 5, -1, 'pending');
Ожидается ошибка: ERROR: Количество блюда в заказе должно быть больше нуля. Передано:
-1
```

```
Корректная вставка
INSERT INTO public.order_item (order_id, dish_id, quantity, cooking_status)
VALUES (2, 5, 2, 'pending');
```

## 4.7. Триггер аудита изменений оклада сотрудника

Триггер срабатывает AFTER UPDATE на таблице employee\_position и при изменении поля salary записывает в таблицу salary\_audit старое и новое значения оклада с временной меткой.

Логирование изменений оклада сотрудника (аудит)

```
Таблица аудита
CREATE TABLE IF NOT EXISTS public.salary_audit (
    audit_id      INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    employee_id   INTEGER NOT NULL,
    old_salary    NUMERIC(10,2),
```

```

        new_salary      NUMERIC(10,2),
        changed_at      TIMESTAMP NOT NULL DEFAULT now()
    );

CREATE OR REPLACE FUNCTION fn_log_salary_change()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Фиксируем только реальные изменения оклада
    IF OLD.salary IS DISTINCT FROM NEW.salary THEN
        INSERT INTO public.salary_audit
            (employee_id, old_salary, new_salary)
        VALUES
            (NEW.employee_id, OLD.salary, NEW.salary);

        RAISE NOTICE 'Аудит: сотрудник id=% - оклад изменён с % на % руб.',
            NEW.employee_id, OLD.salary, NEW.salary;
    END IF;
    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_log_salary_change ON public.employee_position;

CREATE TRIGGER trg_log_salary_change
AFTER UPDATE ON public.employee_position
FOR EACH ROW
EXECUTE PROCEDURE fn_log_salary_change();

```

### Тестирование триггера:

Проверка ДО: таблица аудита пуста  
 SELECT \* FROM public.salary\_audit;

Изменяем оклад сотрудника id=1  
 UPDATE public.employee\_position  
 SET salary = 70000.00  
 WHERE employee\_id = 1 AND end\_date IS NULL;

Проверка ПОСЛЕ: в таблице аудита появилась запись  
 SELECT \* FROM public.salary\_audit;

## 4.8. Триггер проверки даты рождения сотрудника

Триггер срабатывает BEFORE INSERT OR UPDATE на таблице employee и проверяет: дата рождения не может быть в будущем, а возраст сотрудника должен быть не менее 16 лет.

-- ТРИГГЕР 8: Запрет даты рождения сотрудника в будущем

```

CREATE OR REPLACE FUNCTION fn_check_employee_birth_date()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.birth_date >= CURRENT_DATE THEN
        RAISE EXCEPTION
            'Дата рождения сотрудника не может быть в будущем или равна сегодня: %',
            NEW.birth_date;
    END IF;
    RETURN NEW;
END;

```

```

END IF;

-- Возраст не менее 16 лет
IF AGE(CURRENT_DATE, NEW.birth_date) < INTERVAL '16 years' THEN
    RAISE EXCEPTION
        'Сотрудник слишком молод (менее 16 лет). Дата рождения: %',
        NEW.birth_date;
END IF;

RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_check_employee_birth_date ON public.employee;

CREATE TRIGGER trg_check_employee_birth_date
BEFORE INSERT OR UPDATE ON public.employee
FOR EACH ROW
EXECUTE PROCEDURE fn_check_employee_birth_date();

```

### Тестирование триггера:

Попытка добавить сотрудника с датой рождения в будущем  
INSERT INTO public.employee (last\_name, first\_name, birth\_date, phone)  
VALUES ('Тестов', 'Тест', '2030-01-01', '+7-999-000-00-00');  
Ожидается ошибка: ERROR: Дата рождения сотрудника не может быть в будущем...

Попытка добавить несовершеннолетнего (10 лет)  
INSERT INTO public.employee (last\_name, first\_name, birth\_date, phone)  
VALUES ('Юный', 'Сотрудник', CURRENT\_DATE - INTERVAL '10 years', '+7-999-111-11-11');  
Ожидается ошибка: ERROR: Сотрудник слишком молод...

Корректная вставка  
INSERT INTO public.employee (last\_name, first\_name, birth\_date, phone)  
VALUES ('Новиков', 'Павел', '1995-03-20', '+7-999-222-22-22');

## 6. Вывод

В ходе лабораторной работы были разработаны хранимые процедуры, функции и триггеры для базы данных «Ресторан». Процедуры реализуют: вывод заказов заданного официанта на заданную дату, расчёт стоимости заказа по составу позиций и ценам блюд, а также повышение оклада сотрудника на 30% при повышении его категории.

Триггеры обеспечивают: запрет изменения статуса завершённого или оплаченного заказа, автоматический пересчёт стоимости заказа при добавлении или удалении позиций, а также контроль совмещения ролей — официант заказа не может быть одновременно назначен его поваром. Работоспособность всех созданных объектов проверена в SQL Shell psql с использованием SQL-запросов и подтверждающих скриншотов.